Infinite Ocean Surface Rendering

Problem 2
Solution 2
Implementation 3
1. Ocean mesh 3
2. Waves 7
3. Tiling 12
4. Artifacts and mitigation 15
5. Visual enhancements 17
Results 28
Limitations 29
1. Projected grid resolution 29
2. Texture resolution 29
Conclusions 30
How to use the Demo 31
Bibliography 33
Appendix – GLSL Mathematical Functions 34

Oihan Abruña

© DigiPen Institute of Technology Europe - Bilbao

Problem

Rendering a visually convincing infinite ocean surface requires a high level of geometric detail. Wave simulation relies on vertex displacement, which directly depends on the resolution of the underlying mesh.

A naive approach would require an extremely dense mesh to capture fine wave details across the entire surface. However, this is not feasible in a real-time rendering context due to performance constraints. Increasing mesh resolution leads to a significant rise in computational cost.

Additionally, evaluating complex wave functions at every vertex becomes excessively expensive when the mesh is highly subdivided. This creates a clear limitation: achieving high visual fidelity conflicts with the real-time performance requirements.

Solution

To address these challenges, a combination of geometric and procedural techniques is employed.

The mesh is designed to adapt its level of detail based on the camera view, providing higher vertex density in regions close to the viewer while reducing detail in distant areas. This preserves visual quality where it matters most while maintaining efficiency.

Additionally, wave detail is not entirely represented through geometry. Instead, a procedural approach is used to generate tileable wave patterns, allowing complex surface detail to be computed once over a small patch and then reused across the entire ocean surface.

By combining adaptive mesh resolution with reusable procedural wave generation, it is possible to achieve a visually rich ocean surface while satisfying real-time performance constraints.

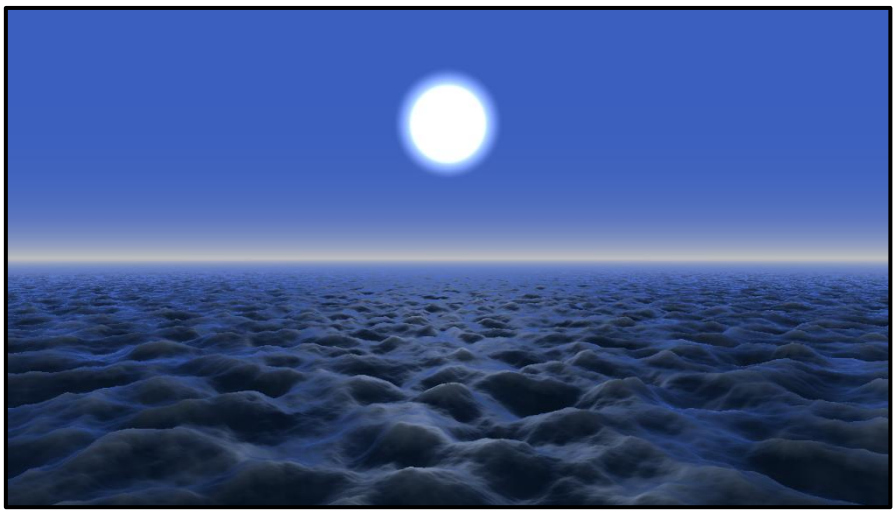

Figure 1: Overview of final ocean surface render.

© DigiPen Institute of Technology Europe - Bilbao

Implementation

1. Ocean mesh

To represent a seemingly infinite water surface, a mesh that covers a large area of the scene is required. This mesh must be sufficiently subdivided to provide enough geometric detail for vertex displacement, which is essential for simulating waves and achieving a visually convincing ocean surface.

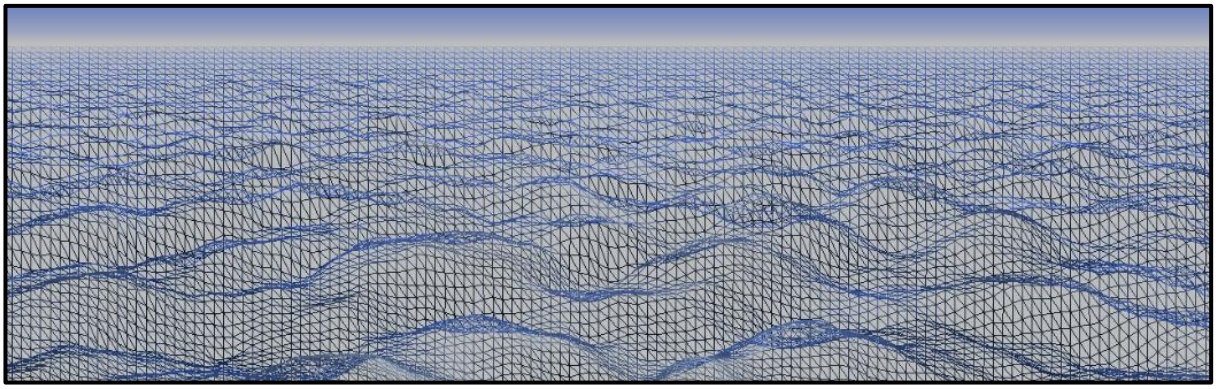

Figure 2: Overview of final ocean mesh in wireframe mode.

However, using a uniformly dense mesh across the entire surface is not feasible due to performance constraints. Therefore, an efficient strategy for distributing geometric detail is necessary.

1.1. View-dependent detail

The detail required varies depending on the distance to the camera. Regions close to the viewer occupy more screen space and require higher geometric resolution, while distant regions contribute less to the final image and can be represented with fewer vertices.

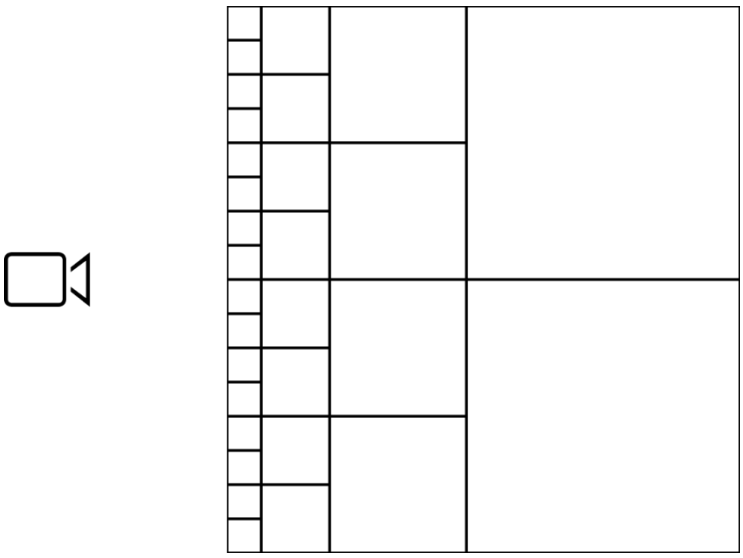

Figure 3: View-dependent mesh resolution. Dense near the camera, sparse in the distance.

This makes it necessary to distribute vertices non-uniformly, concentrating detail near the camera while reducing it farther away.

© DigiPen Institute of Technology Europe - Bilbao

1.2. Alternative approaches

A possible way to construct an ocean surface is to define the mesh in world space, where the geometry exists as part of the scene independently of the camera. In this case, the challenge becomes how to distribute the vertices of this mesh efficiently across a very large area while maintaining sufficient detail near the viewer.

Several techniques have been proposed to address this problem:

1.2.1. Multi-Resolution Ring Meshes

One approach is to construct a large mesh composed of concentric rings with different levels of detail. Inner regions, closer to the camera, are highly subdivided, while outer regions progressively reduce their resolution.

This method provides a straightforward way to distribute detail over a large surface. However, it introduces challenges in ensuring smooth transitions between levels of detail. Without careful handling, visible seams or discontinuities may appear between adjacent regions.

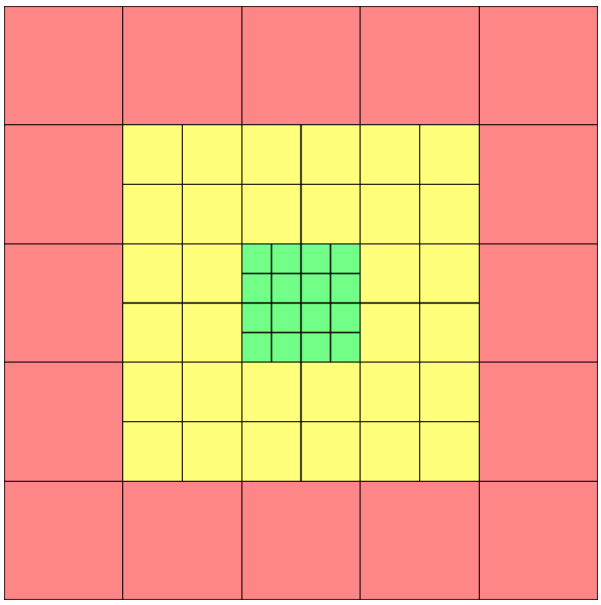

Figure 4: Example of a ring-based mesh with decreasing resolution from inner (green) to outer regions (red).

1.2.2. Hardware Tessellation

Another approach is to use hardware tessellation to dynamically subdivide the mesh based on the distance to the camera, which allows adaptive refinement at runtime.

Despite being supported by modern GPUs, tessellation can become computationally expensive when high levels of subdivision are required, particularly in large-scale scenes such as an ocean surface.

© DigiPen Institute of Technology Europe - Bilbao

1.3. Adopted approach: Projected grid

For this implementation, a different strategy is used based on the projected grid technique (Johanson, 2004). This method constructs the mesh, whose vertices are even spaced, in post-perspective camera space rather than in world space.

By doing so, the resulting geometry naturally provides higher resolution in regions close to the viewer and lower resolution in distant areas, achieving spatial scalability without requiring multiple levels of detail.

To obtain the final mesh, the grid is projected back into world space by applying the inverse of the camera's view and projection transformations.

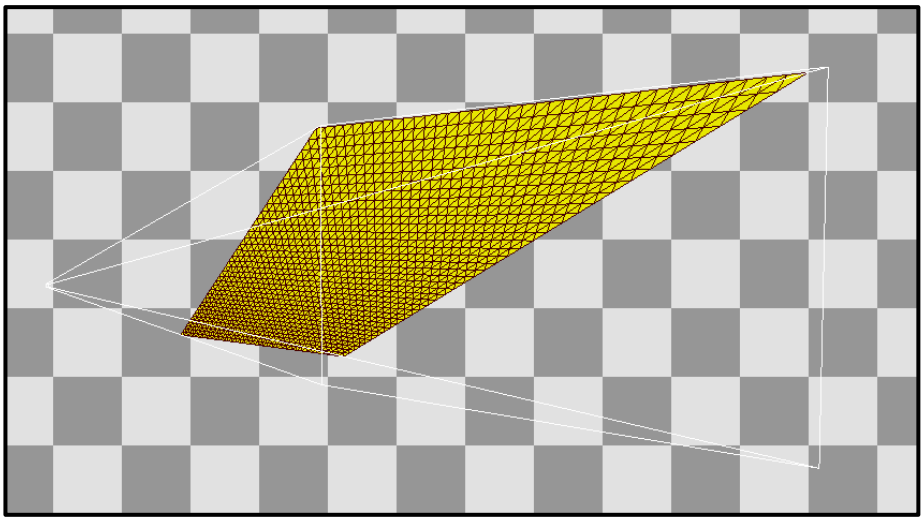

Figure 5: Visualization of a projected grid within the camera frustum from an auxiliary camera.

1.3.1. Grid creation

The first step is to create a regular, discretized plane in post-perspective space that is oriented orthogonally to the camera's view direction. This defines a uniform grid relative to the screen, ensuring even spacing in image space.

The resolution of the grid can be adjusted according to the desired level of detail.

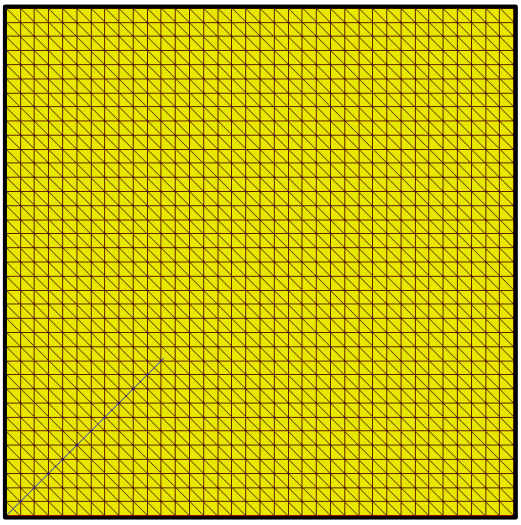

Figure 6: Example of an input projected mesh in the unit square [0, 1].

© DigiPen Institute of Technology Europe - Bilbao

1.3.2. Projection to world space

Once the grid is defined in post-perspective space, it is projected back onto a plane in world space, using the perspective projection of the camera. Distant grid cells correspond to much larger regions in world space than near cells. As a result, this projection naturally concentrates geometric detail near the camera.

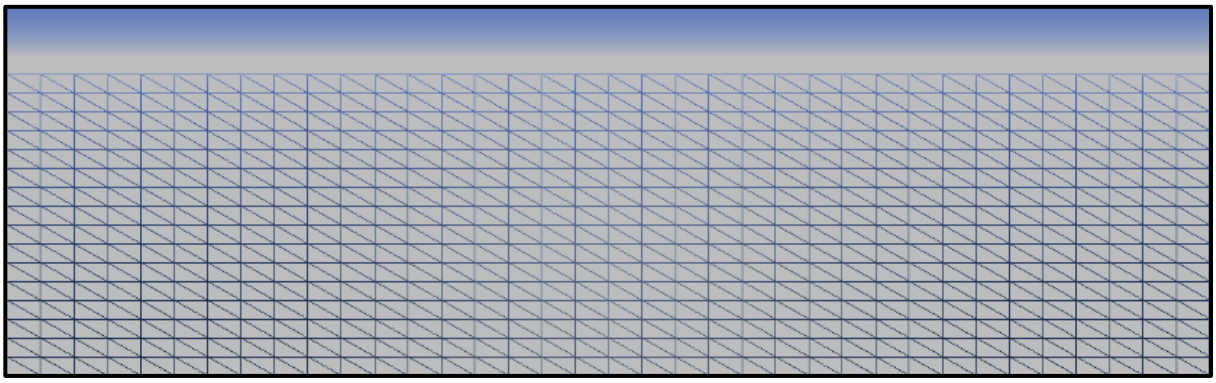

Figure 7: Example of a projected mesh on a world-space plane.

Let $Q \in [0,1]^2$ be a vertex coordinate in the grid. First the grid coordinate should be converted to normalized device coordinates (NDC) in the range $[-1,1]$:

$$Q_{\text{ndc}} = 2Q - 1$$

A ray from the camera through the far plane is defined by lifting Q_{ndc} to $(\text{Z} = 1)$:

$$Q_{\text{far}} = \begin{bmatrix} Q_{\text{ndc}_x} \\ Q_{\text{ndc}_y} \\ 1 \\ 1 \end{bmatrix}$$

This point is converted to view space and then world space by applying the inverse projection P^{-1} and inverse view V^{-1} transformations:

$$Q_{\text{v}} = P^{-1}Q_{\text{far}}, \qquad Q_{\text{v}} = \frac{Q_{\text{v}}}{Q_{\text{v}_w}}, \qquad Q_{\text{w}} = V^{-1}Q_{\text{v}}$$

The ray direction from the camera position C is:

$$d = \frac{Q_{\text{w}} - C}{\|Q_{\text{w}} - C\|}$$

Assuming the ocean plane is horizontal and aligned with the world up vector (y-axis) and located at $y = 0$, the intersection parameter along the ray is:

$$t = -\frac{C_y}{d_y}$$

The world-space position on the plane, moving non-intersecting rays to the horizon, is:

$$W = \begin{cases} C + td, & t \geq 0 \\ \begin{bmatrix} Q_{\text{w}_x} \\ 0 \\ Q_{\text{w}_z} \end{bmatrix}, & t < 0 \end{cases}$$

© DigiPen Institute of Technology Europe - Bilbao

2. Waves

Ocean waves are represented as displacements of the ocean mesh vertices. In principle, wave equations could be evaluated directly in the vertex shader. However, computing these equations per vertex across a large projected grid does not scale efficiently.

To address this, tileable procedural textures are generated. By computing the wave displacements for a single patch of the ocean and storing the results in textures, the same patch can be reused by tiling it across the entire surface, greatly reducing computational cost while preserving visual detail.

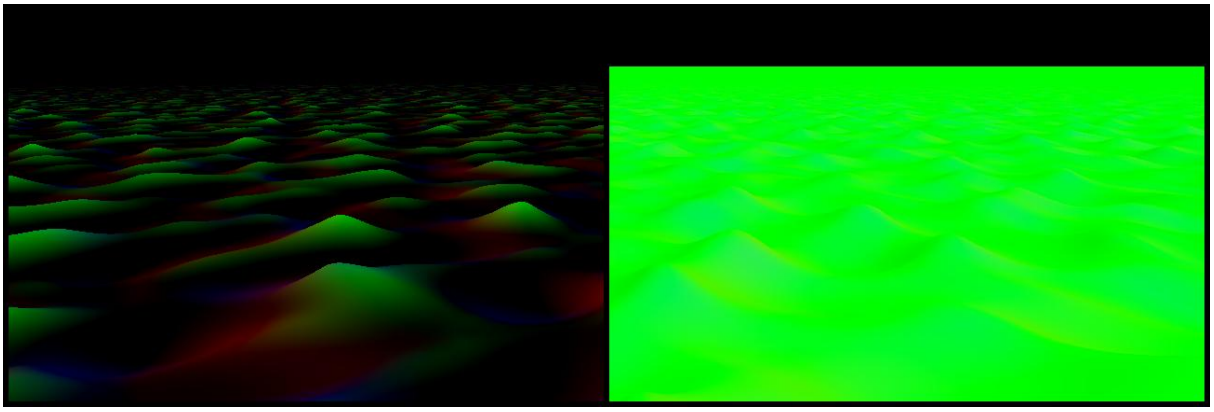

Figure 8: Displaced ocean mesh with world-space displacement as colors (left) and world-space normals as colors (right).

2.1. Procedural texture generation

The computation of the wave textures is highly parallelizable and is performed entirely on the GPU using a compute shader. Two textures are generated:

1. Displacement texture: stores vertex displacement values in world space, used in the vertex shader to offset vertex positions.
2. Normal texture: stores the surface normals in world space, used in the fragment shader for lighting calculations.

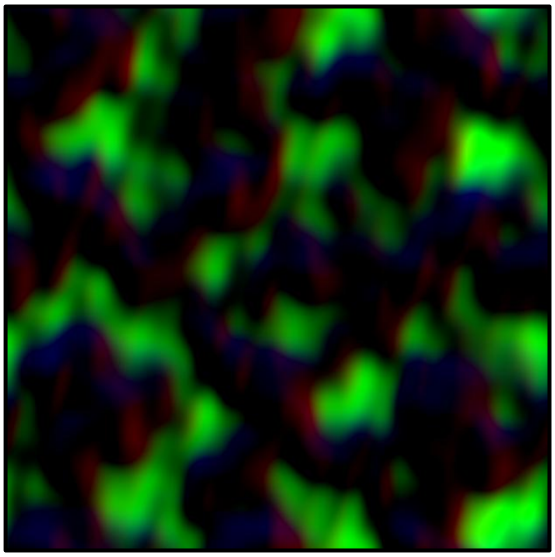

Figure 10: Example of displacement map in world space.

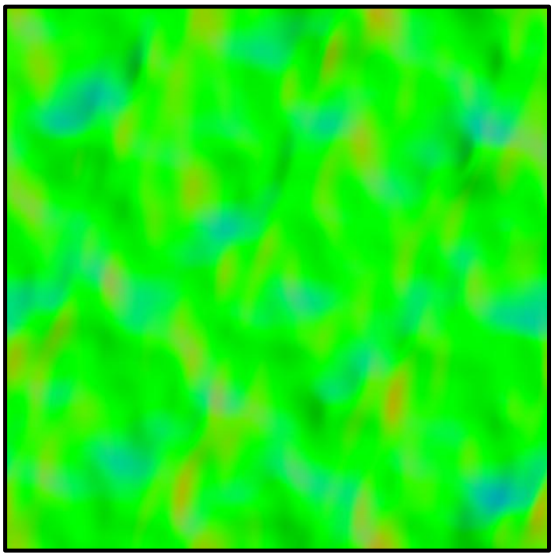

Figure 9: Example of normal map in world space.

© DigiPen Institute of Technology Europe - Bilbao

2.2. Alternative approach: FFT

An alternative method for generating ocean wave textures is to use spectral synthesis via Fast Fourier Transform (FFT) (Tessendorf, 2001). In this approach, the displacement map is constructed in frequency domain and then transformed into the spatial domain using the FFT.

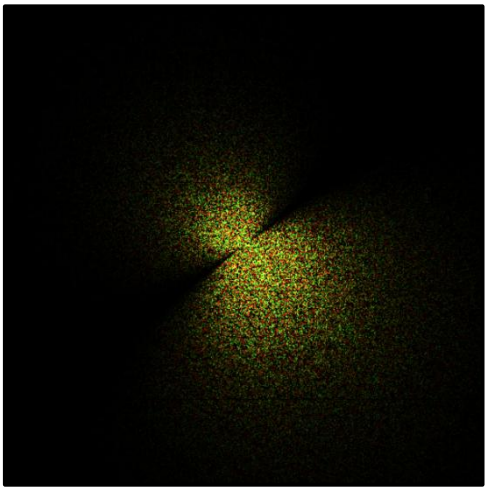

Figure 11: Frequency-domain representation of ocean waves (NVIDIA Corporation).

To generate displacement maps for the three spatial directions (x, y, z) three separate spectra are maintained at runtime. Each spectrum is then transformed into the spatial domain using the inverse FFT, producing the corresponding displacement maps in world space.

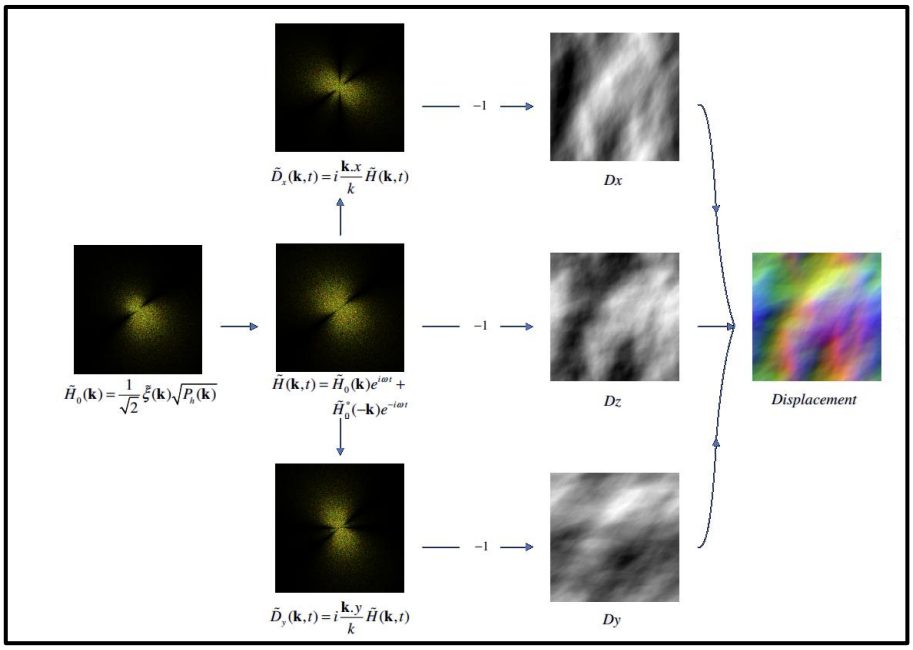

Figure 12: Diagram of spectrum transformation process (NVIDIA Corporation).

The FFT-based method naturally produces tileable textures, which can be directly repeated across the ocean surface without introducing discontinuities.

However, in this implementation, a different procedural approach is used, which does not inherently generate tileable textures. Therefore, special care must be taken to ensure the generated textures can be seamlessly tiled across the projected grid.

© DigiPen Institute of Technology Europe - Bilbao

2.3. Adopted approach: Sum of Gerstner waves

In this implementation, waves are represented as a sum of Gerstner or trochoidal waves.

2.3.1. Gerstner waves

Gerstner waves are an analytical solution for periodic surface gravity waves and describe a wave of permanent form on the surface of an incompressible fluid of infinite depth (Wikipedia contributors, n.d.).

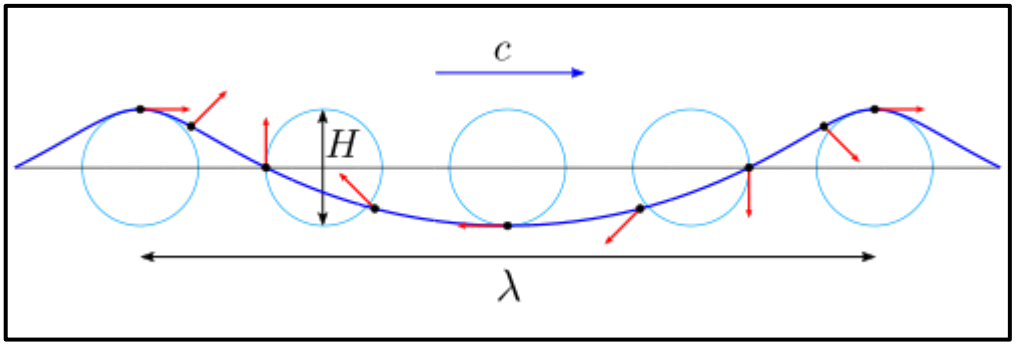

Figure 13: Surface elevation of a trochoidal wave (Wikipedia contributors, n.d.).

The Gerstner wave function defines the displacement of the surface, and its derivatives can be used to compute normals (Finch, 2004).

With a total number of summed waves N, for a single wave i, let:

- A_i: amplitude.
- λ_i: wavelength.
- $k_i = \frac{2\pi}{\lambda_i}$: wave number.
- $\omega_i = \sqrt{gk_i}$: angular frequency.
- $\overrightarrow{d_i} = (d_{x,i}, d_{z,i})$: propagation normalized direction in the horizontal plane.
- Q_i: steepness parameter.
- ϕ_i: phase offset.

The phase θ of the wave at $\mathrm{P} = (x, z)$ is:

$$\theta_i = k_i \, \mathrm{d}_i \cdot \mathrm{P} - \omega_i t + \phi_i$$

The displacement contributed by wave i is:

$$\Delta x_i = Q_i d_{x,i} cos(\theta_i)$$

$$\Delta z_i = Q_i d_{z,i} cos(\theta_i)$$

$$\Delta y_i = A_i sin(\theta_i)$$

The total displacement is obtained by summing all wave contributions:

$$\vec{o} = \sum_{i=1}^{N} \begin{bmatrix} \Delta x_i \\ \Delta y_i \\ \Delta z_i \end{bmatrix}$$

© DigiPen Institute of Technology Europe - Bilbao

This implementation slightly deviates from the classical Gerstner formulation. The horizontal displacement is controlled exclusively by the steepness Q_i, and does not depend on the amplitude A_i.

- When $Q_i = 0$, the wave is a simple sine wave with no horizontal displacement.
- When $\sum_{i=1}^{N} Q_i k_i = 1$, the wave reaches its maximum non-self-intersecting form.
- When $\sum_{i=1}^{N} Q_i k_i > 1$ the surface may develop loops near wave crests.

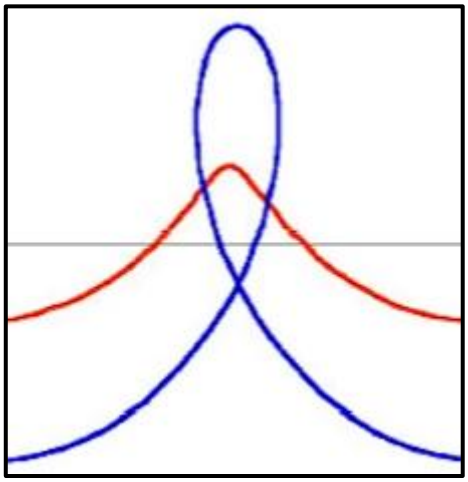

Figure 14: Gerstner wave creating loop.

To avoid such artifacts while still allowing a range from smooth to sharp waves, it is recommended to define a sharpness parameter $S_i \in [0,1]$. The steepness is defined as $Q_i = \frac{S_i}{k_i}$, such that $Q_i k_i = S_i$, and therefore the loop condition simplifies to $\sum_{i=1}^{N} S_i \leq 1$. A global scaling factor is applied to enforce this constraint:

$$\mathrm{S} = \sum_{i=1}^{N} S_i\,, \qquad f = \frac{1}{\max(\mathrm{S},\ 1)}, \qquad Q_i = \frac{S_i}{k_i} f$$

The surface normal is computed from the derivatives of the displaced surface. The Gerstner displacement function is easily differentiable, allowing accurate and stable normal computation. The tangent space basis vectors are obtained from the partial derivatives of the displaced surface with respect to the horizontal coordinates:

$$\vec{B} = \begin{bmatrix} 1 - \sum_{i=1}^{N} Q_i d_{x,i}^2 k_i sin(\theta_i) \\ \sum_{i=1}^{N} d_{x,i} k_i A_i cos(\theta_i) \\ -\sum_{i=1}^{N} Q_i d_{x,i} d_{z,i} k_i sin(\theta_i) \end{bmatrix} \qquad \vec{T} = \begin{bmatrix} -\sum_{i=1}^{N} Q_i d_{x,i} d_{z,i} k_i sin(\theta_i) \\ \sum_{i=1}^{N} d_{z,i} k_i A_i cos(\theta_i) \\ 1 - \sum_{i=1}^{N} Q_i d_{z,i}^2 k_i sin(\theta_i) \end{bmatrix}$$

$$\vec{N} = \frac{\vec{T} \times \vec{B}}{\| \vec{T} \times \vec{B} \|}$$

© DigiPen Institute of Technology Europe - Bilbao

2.3.2. Multi-Scale Random-Direction Wave Summation

To add realistic detail across multiple scales, the final surface displacement is computed as the sum of multiple Gerstner waves, each with progressively smaller amplitude and wavelength.

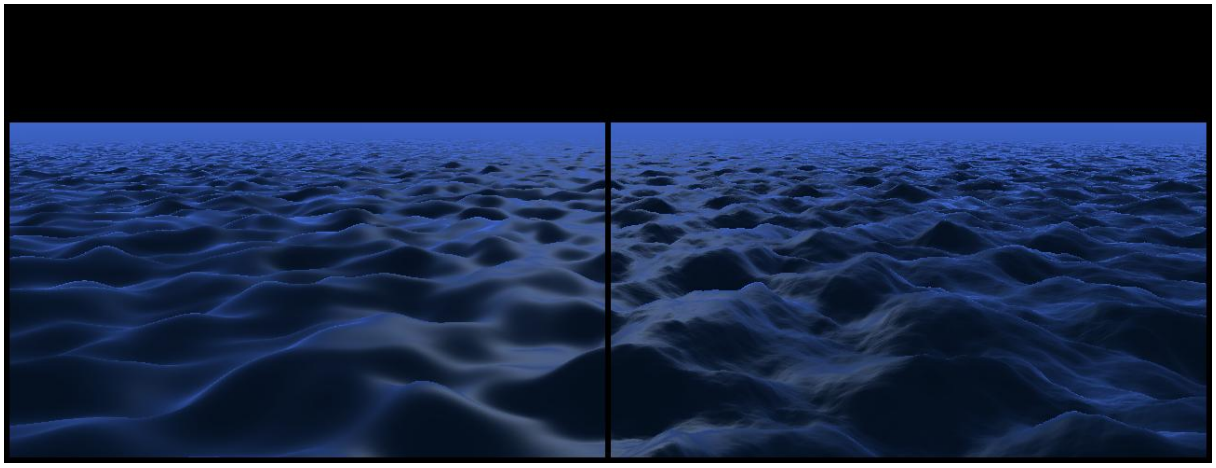

Figure 15: Low wave count (16 waves, left) vs. high wave count (64 waves, right).

Let $i = 0, \dots, N-1$ index the waves. Starting from initial parameters A_0, λ_0, and S_0, the amplitude, wavelength, and steepness of each successive wave are scaled relative to the previous one:

$$\begin{aligned} A_{i+1} &= A_i \cdot f_A \\ \lambda_{i+1} &= \lambda_i \cdot f_\lambda \\ S_{i+1} &= S_i \cdot f_S \end{aligned}$$

where f_A, f_λ, f_S are scaling factors controlling the decay of these wave parameters.

To increase visual variation, each Gerstner wave is assigned a random horizontal direction. A uniform distribution of directions is desirable to avoid visible patterns or alignment artifacts. This is achieved by sampling a random angle θ_i within a configured range $[\theta_{min}, \theta_{max}]$ for each wave. The corresponding horizontal direction vector is then computed as:

$$\vec{d_i} = \begin{bmatrix} \cos(\theta_i) \\ \sin(\theta_i) \end{bmatrix}$$

By generating θ_i uniformly, the waves are evenly distributed in direction, producing a natural and varied surface appearance.

© DigiPen Institute of Technology Europe - Bilbao

3. Tiling

To efficiently render ocean surfaces, wave textures derived from Gerstner waves must be repeated across a tileable patch. Careful handling is required to preserve seamless tiling.

3.1. Preserving tiling

For Gerstner waves to preserve tiling, the wave phase must be periodic across the tile. This happens when the wavelength fits exactly into the tile size along the wave's direction.

Figure 16: Seamless tiling of the ocean patch.

For seamless tiling over a square patch of size L, the phase must match at opposite edges of the tile:

$$\theta_i(x + L, z) - \theta_i(x, z) = 2\pi n_x$$

$$\theta_i(x, z + L) - \theta_i(x, z) = 2\pi n_z$$

Substituting the phase, we obtain the necessary conditions:

$$k_i\, d_{x,i} L = 2\pi n_x \qquad k_i\, d_{z,i} L = 2\pi n_z \qquad n_x, n_z \in \mathrm{Z}^+$$

The wavelength must be a divisor of the tile size along the wave direction:

$$\lambda_i = \frac{L}{n} \qquad n \in \mathrm{Z}^+$$

Additionally, continuous directions cannot be used directly when computing the wave phase, as this would break tileability. This restricts the use of normalized direction vectors. Instead, integer-valued directions are required to ensure discrete phase increments that preserve periodic tiling. To achieve this, the sampled continuous direction is projected onto the integer lattice using rounding, with a maximum bound d_{max} imposed on the components of the direction vector:

$$\vec{d} = round(d_{max} \cdot \overrightarrow{d_i})$$

This approach has some drawbacks, since using non-normalized directions indirectly scales the wavelength. Consequently, increasing directional variety requires allowing larger integer direction vectors, which must be carefully balanced against the tile size to maintain visual consistency.

© DigiPen Institute of Technology Europe - Bilbao

3.2. Sampling textures

To ensure consistent tiling across the ocean surface, texture coordinates are derived directly from the world-space position. This guarantees that textures repeat seamlessly over the entire domain, independent of mesh resolution or deformation.

Given a world-space position W, and assuming the ocean plane is horizontal and is aligned with the world up vector (y-axis), the texture coordinates are computed as:

$$\mathrm{UV} = \frac{W_{xz}}{L}$$

where W_{xz} denotes the horizontal components of the position and L is the tile size of the tile patch.

When texture wrapping is set to repeat, the resulting coordinates are implicitly mapped into the $[0, 1]$ domain by taking their fractional part:

$$\mathrm{UV}_{tile} = fract\left(\frac{W_{xz}}{L}\right)$$

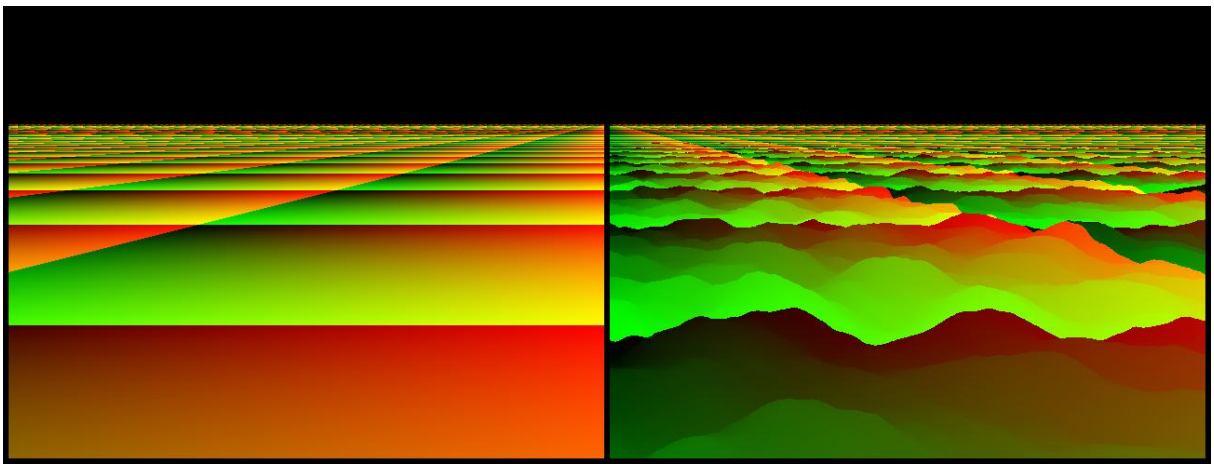

Figure 17: Texture tiling using UVs as colors, before displacement (left) and after wave displacement (right).

The choice of texture filtering plays a crucial role in maintaining both visual quality and performance. It is recommended to use trilinear filtering with mipmaps, as this helps hide aliasing and sampling artifacts. These artifacts are particularly noticeable at grazing angles, which commonly occur when the camera is close to the surface and looking toward the horizon, where high-frequency detail can cause shimmering and instability.

Furthermore, this implementation relies heavily on texture sampling. Sampling high-resolution textures when they occupy only a small portion of the screen is inefficient. By using lower resolution levels when appropriate, the sampling process becomes significantly more efficient while preserving visual coherence.

However, this optimization is not directly available in the vertex shader, where explicit mipmap level selection is not supported. Although the appropriate level could be approximated, this implementation instead samples the displacement map using only the original texture resolution for the sake of simplicity.

© DigiPen Institute of Technology Europe - Bilbao

3.3. Remove visible tiling pattern

Tiling introduces visible repeating patterns. To eliminate these, a method that applies a triangle lattice in UV space is used to blend overlapping texture samples (Thomas & Eric, 2019), breaking repetition while maintaining continuity.

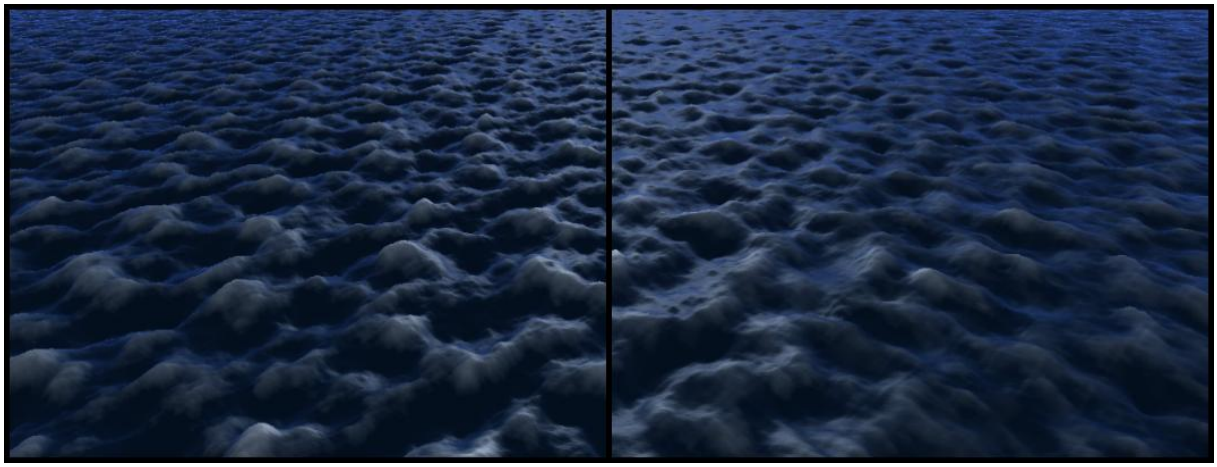

Figure 18: Standard tiling pattern visible (left) vs. tiling pattern mitigated using triangle-grid blending (right).

Tiling

The UV domain is first scaled by a factor of $2\sqrt{3}$, which controls the size of the input with respect to the size of the tiles. This parameter can be adjusted depending on the characteristics of the input texture.

The coordinates are then skewed into an equilateral triangle grid, which partitions the UV space into non-overlapping triangles. For each UV point, the containing triangle is identified, and barycentric coordinates (w_1, w_2, w_3) relative to the triangle vertices are computed.

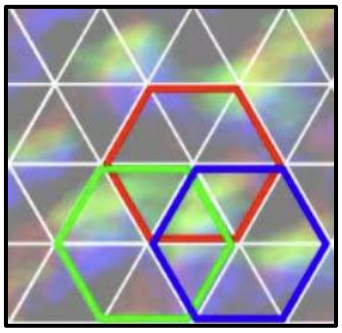

Figure 19: Triangular lattice used for procedural tiling-and-blending.

Blending

Each triangle vertex is assigned a pseudo-random offset via a hash of its integer ID. These offsets are used to fetch different regions of the input texture, introducing stochastic variation across the surface. The final value is computed by linear interpolation of the three sampled texture values I_1, I_2, I_3, weighted by the barycentric coordinates:

$$I = w_1 I_1 + w_2 I_2 + w_3 I_3$$

While linear blending is a simplification relative to more advanced methods, it is sufficient in this implementation to suppress noticeable tiling artifacts.

© DigiPen Institute of Technology Europe - Bilbao

4. Artifacts and mitigation

When combining the projected grid method with Gerstner wave displacement, several visual artifacts may arise due to the dynamic modification of vertex positions.

4.1. Gaps at Screen Boundaries

The projected grid method works correctly for a flat plane, where the vertices perfectly fit the camera's view frustum. However, when vertices are displaced by wave functions it can result in gaps along the edges of the screen, where the displaced ocean no longer fully covers the view, leading to visible artifacts.

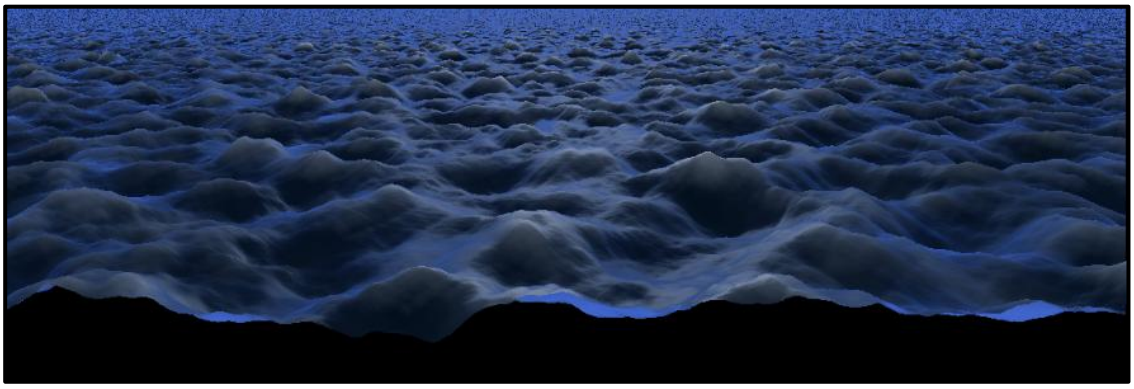

Figure 20: Example of screen-space gaps caused by displaced vertices, intentionally exaggerated for clarity.

To mitigate this, the plane is offset along the view direction by a configurable parameter. This offset expands the grid in normalized device coordinates (NDC) before projection:

$$Q_{\text{ndc}} = (2Q - 1) \cdot offset$$

By scaling the NDC coordinates with $offset > 1$ the projected grid covers a larger portion of the view plane, ensuring that the displaced vertices cover the screen fully.

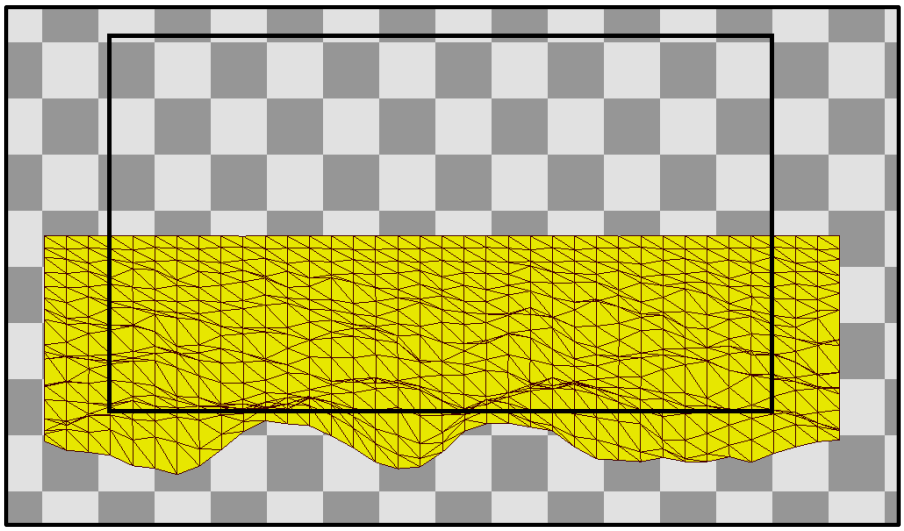

Figure 21: Mesh output showing the plane offset, ensuring displaced vertices fully cover the screen and eliminating boundary gaps.

© DigiPen Institute of Technology Europe - Bilbao

4.2. Horizon Aliasing

Although aliasing artifacts are already reduced with the use of trilinear filtering and mipmaps, additional measures are recommended to further improve stability in distant regions.

In regions near the horizon, the projected grid produces a very low vertex density due to the effects of perspective projection. When high-frequency wave components are sampled on this sparse geometry, aliasing artifacts appear. These are perceived as noise or shimmering on the ocean surface, particularly noticeable during camera movement.

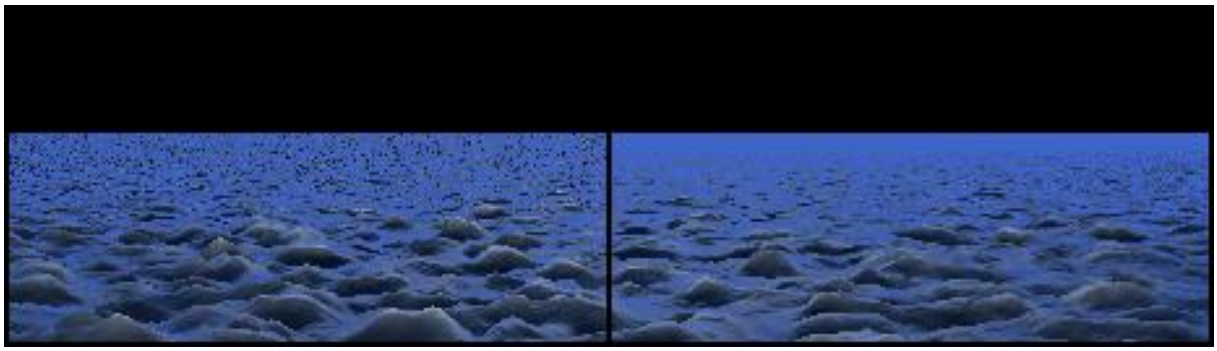

Figure 22: Horizon aliasing (left) vs. mitigated result (right).

To mitigate this issue, distance-based attenuation is applied. As the distance to the camera increases, the wave displacement is progressively reduced.

Let W be the world-space position of the vertex and C the camera position. The distance to the camera is:

$$d = \| W - C \|$$

A fade factor is defined based on distance, where $d_{\max}$ is the maximum distance at which displacement is applied:

$$f = clamp\left(1 - \frac{d}{d_{max}}, 0, 1\right)$$

This fade factor is used both to attenuate the displacement and to smooth the surface normals.

Given a displacement vector $\vec{o}$, the attenuated displacement is:

$$\overrightarrow{o'} = f \cdot \vec{o}$$

Similarly, the surface normal is blended with the up vector $\vec{u} = (0, 1, 0)$:

$$\overrightarrow{n'} = \frac{(1-f) \cdot \vec{u} + f \cdot \vec{n}}{\| (1-f) \cdot \vec{u} + f \cdot \vec{n} \|}$$

This limits the contribution of high-frequency details in distant regions, thereby reducing visible aliasing.

© DigiPen Institute of Technology Europe - Bilbao

5. Visual enhancements

To improve realism, additional techniques are introduced to enhance not only the appearance of the water surface, but also the rendering of the surrounding atmosphere and submerged objects, including scenarios where the camera is located underwater.

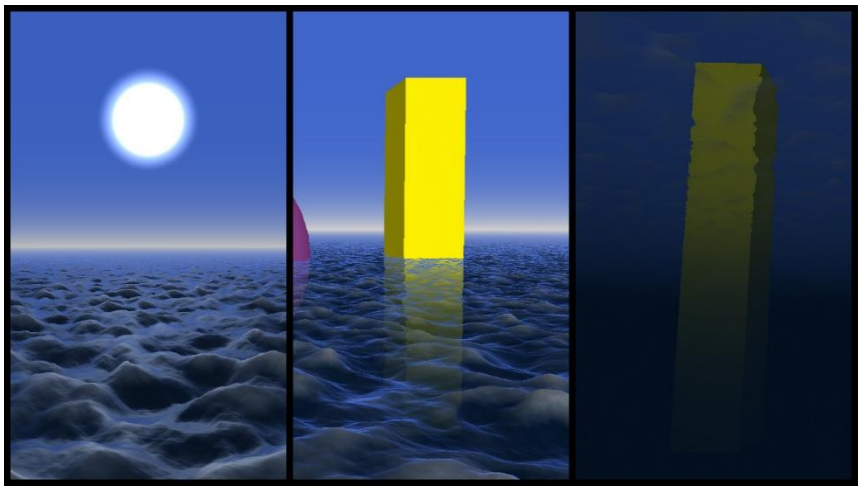

Figure 23: Preview of the enhanced ocean rendering.

5.1. Starting point

While the current implementation is functional, it provides limited visual fidelity and does not convey realistic behavior and interaction with light. At this stage, the ocean appears more like a simple displaced mesh than a dynamic, immersive body of water.

5.1.1. Above water

When the camera is positioned above the water surface, both the ocean and the sky are rendered using constant colors.

This approach serves as a basic visual reference for the water plane and the horizon. However, it lacks important optical effects such as reflections, refractions, and color variation, resulting in a flat and unrealistic appearance.

Figure 24: Output image of ocean surface without visual enhancements.

© DigiPen Institute of Technology Europe - Bilbao

5.1.2. Submerged objects

Objects partially or fully immersed in the water are seen through the water surface. There is no proper interaction between light and the water medium.

Light passes through the water without attenuation, and no distortion or scattering is applied. As a result, the water appears perfectly transparent, and submerged objects retain their original color and brightness regardless of depth.

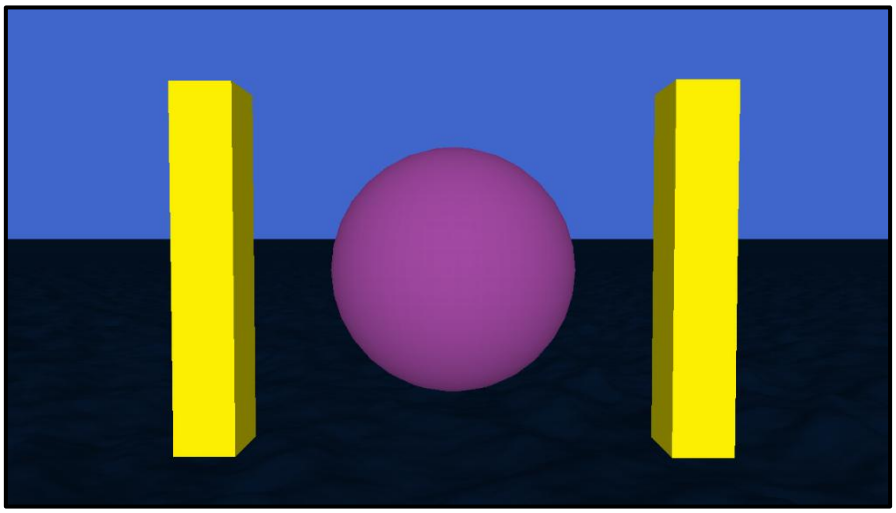

Figure 25: Output image of submerged objects without visual enhancements.

5.1.3. Underwater view

When the camera moves below the water surface, the entire surrounding medium is rendered using the same constant color as the surface. Submerged objects have the same issues described previously.

Additionally, the water surface itself is not visible from below, as lighting is only considered from above and no reflective or refractive effects are computed. This creates the impression of a perfectly transparent medium.

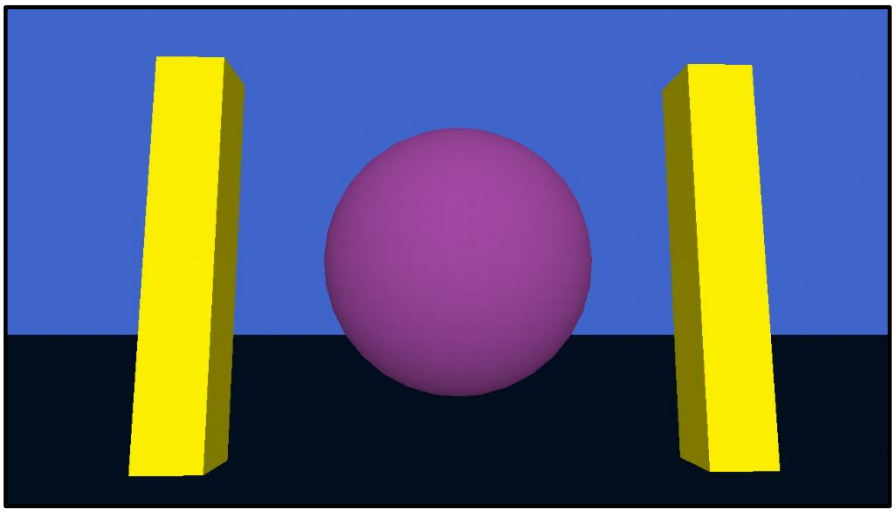

Figure 26: Output image of underwater view without visual enhancements.

© DigiPen Institute of Technology Europe - Bilbao

5.2. Foam

Foam is generated based on the local deformation of the water surface and tends to appear in regions associated with wave crests and wave breaking (Tessendorf, 2001).

This behavior can be modeled using the Jacobian matrix of the horizontal displacement, which contains the partial derivatives of the displaced surface coordinates (x', z') with respect to the original coordinates (x, z). These partial derivatives correspond to the horizontal components of the surface tangent space vectors already computed for normal evaluation:

$$J = \begin{bmatrix} \frac{\partial x'}{\partial x} & \frac{\partial x'}{\partial z} \\ \frac{\partial z'}{\partial x} & \frac{\partial z'}{\partial z} \end{bmatrix} = \begin{bmatrix} B_x & T_x \\ B_z & T_z \end{bmatrix}$$

The determinant of this matrix is computed as:

$$\det(J) = B_x T_z - B_z T_x$$

This value represents the local area change of the surface. Values < 1 are commonly associated with wave breaking regions where foam and spray would naturally occur.

$$F_{new} = k \cdot clamp(1 - \det(J), 0, 1)$$

To ensure temporal coherence, foam is accumulated over time while gradually decaying:

$$F_{new} = F_{old} \cdot d + F_{new}$$

A foam map is generated to store and accumulate foam intensity over time. This method produces foam that naturally appears at wave crests and fades smoothly over time, enhancing the realism of the simulated ocean surface.

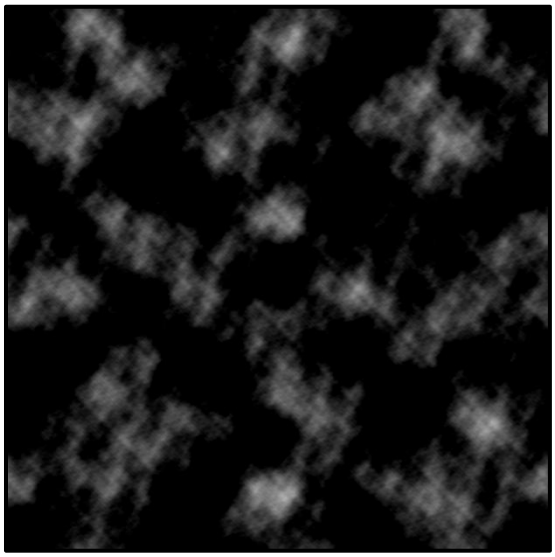

Figure 27: Example of foam map.

Figure 28: Output image of ocean surface with foam.

© DigiPen Institute of Technology Europe - Bilbao

5.3. Fog

Fog is applied as a post-processing effect to hide artifacts near the horizon. An exponential model is used, combining two components:

1. Distance-based fog: increases with the distance from the camera.

$$f_d = clamp\left(1 - e^{-(d-d_0)\cdot\alpha_d}, 0, 1\right)$$

where:

- d: world-space distance from the camera.
- d_0: fog start distance.
- α_d: fall-off based on distance.

2. Height-based fog: decreases with elevation above the water surface.

$$f_h = clamp(e^{-h\cdot\alpha_h}, 0, 1)$$

where:

- h: world-space height.
- α_h: fall-off based on height.

These two components are combined to compute the final fog amount:

$$f = f_d \cdot f_h$$

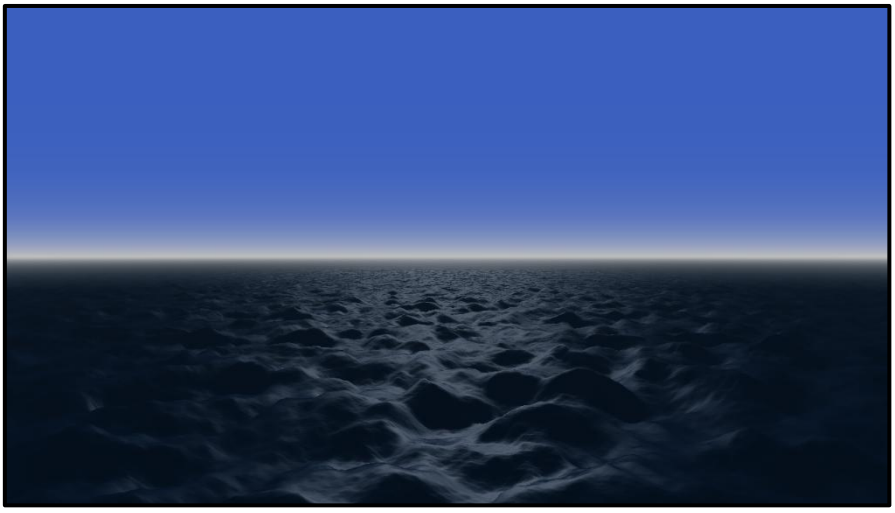

Figure 29: Output image of ocean surface with fog.

© DigiPen Institute of Technology Europe - Bilbao

5.4. Reflections

Reflections on the water surface are approximated using the Schlick approximation of the Fresnel term (Schlick, 1994), which modulates the contribution of reflections based on the viewing angle. The Fresnel factor is computed as:

$$F = F_0 + (1 - F_0) \cdot (1 - cos\theta)^5$$

where:

- F_0: base reflectivity.
- θ: angle between the surface normal and the view direction.

This approach emphasizes reflections at glancing angles, while minimizing reflection when looking directly down at the surface.

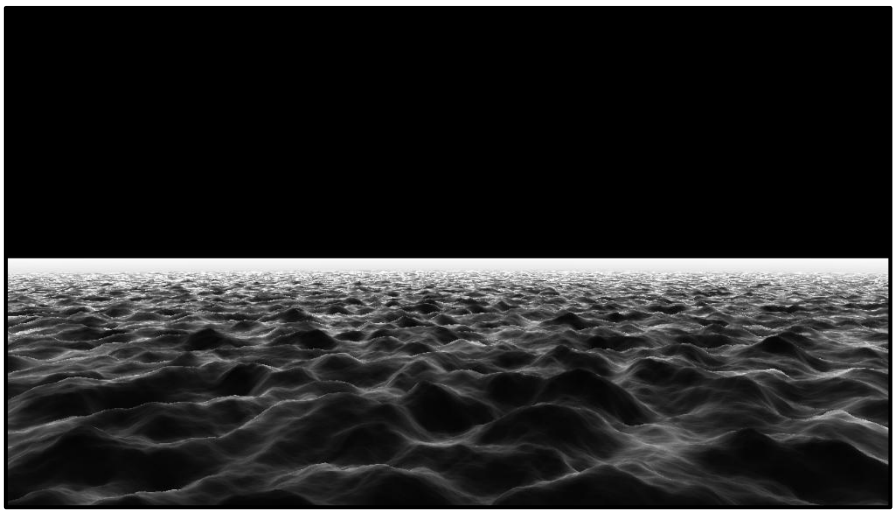

Figure 30: Output image of ocean surface showing reflectance.

In this implementation, reflections are approximated using a constant sky color. This could be improved by replacing it with an environment map to achieve more realistic and spatially varying reflections.

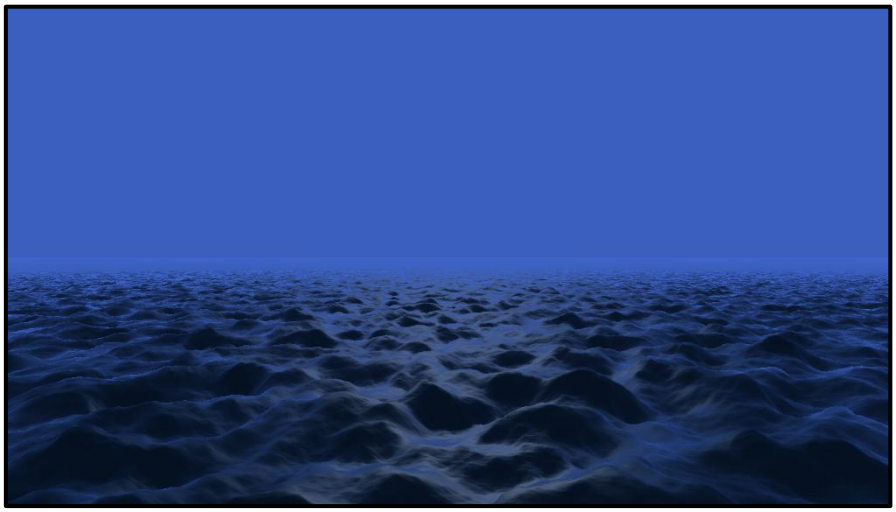

Figure 31: Output image of ocean surface with sky reflections.

© DigiPen Institute of Technology Europe - Bilbao

5.5. Refraction

Refraction is approximated by offsetting screen-space texture coordinates to sample the scene beneath the water surface.

To simulate light bending, the distance between the world-space position of the underlying scene P_s and the water surface P_w is computed. Larger distances correspond to stronger offsets, reflecting the longer path of light through the water.

This effect is modeled exponentially:

$$d = \| P_s - P_w \|$$

$$f_{\text{refract}} = 1 - e^{-\text{d}\cdot s}$$

where:

- s: strength of the refraction.

The refracted UV coordinates used to sample the scene texture are then computed as:

$$\text{UV}_{\text{refract}} = \text{UV}_{\text{screen}} + N_{xy} \cdot f_{\text{refract}}$$

where:

- N: view-space normal of the water surface.

Using N_{xy} in view space ensures the offset is aligned with the camera's screen axes. N_x controls the horizontal shift and N_y controls the vertical shift.

This method is not physically accurate but provides an acceptable visual deformation effect, conveying the appearance of light bending through water.

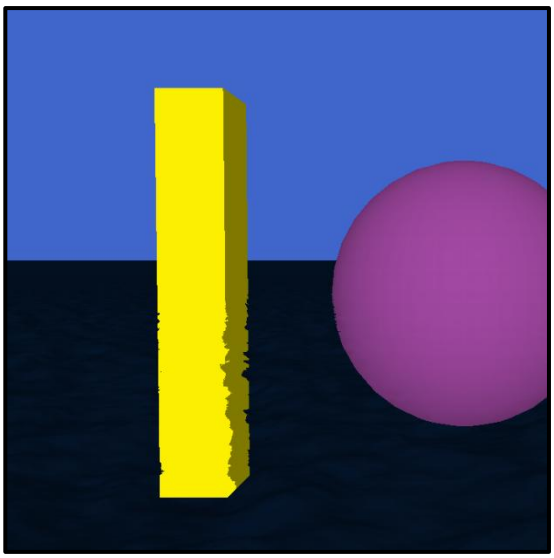

Figure 33: Output image of ocean surface with refraction.

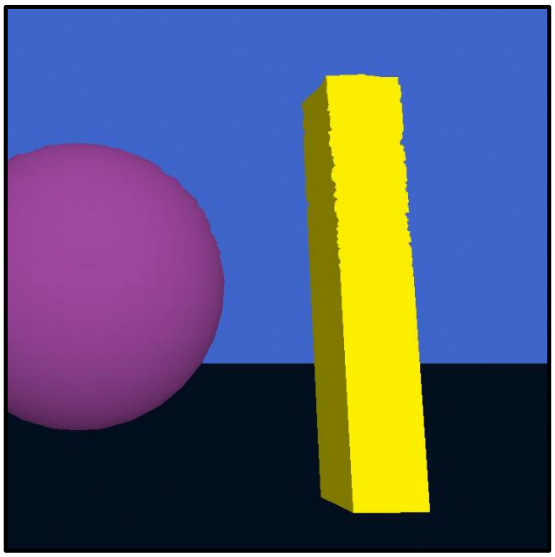

Figure 32: Output image of underwater view with refraction.

© DigiPen Institute of Technology Europe - Bilbao

5.6. Underwater effect

When objects are below the water surface, an underwater effect is applied to simulate light attenuation and create the appearance of submersion. Two scenarios must be considered:

1. The camera is outside the water, so the effect is applied when blending the water surface with the scene geometry underneath.
2. The camera is inside the water, so the effect is applied as a post-processing effect.

Similarly to fog, an exponential model is used, combining two components:

1. Distance-based absorption: depends on the horizontal distance.

$$f_{\mathrm{h}} = \mathrm{clamp}(1 - e^{-d \cdot \alpha_h}, 0, 1)$$

where:

- d: horizontal world-space distance from the water surface (1) or camera (2).
- α_h: fall-off based on distance.

2. Depth-based absorption: simulates light attenuation with increasing depth. Assuming the water plane is at y = 0:

$$depth = -\min(W_y, 0)$$

$$f_{\mathrm{d}} = \mathrm{clamp}(1 - e^{-depth \cdot \alpha_d}, 0, 1)$$

where:

- W_y: world-space height.
- α_d: fall-off based on depth.

Multiplying the surviving fractions gives the portion of light that reaches the camera after passing through both effects. The total light absorbed can then be computed as:

$$f = 1 - (1 - f_{\mathrm{h}}) \cdot (1 - f_{\mathrm{d}})$$

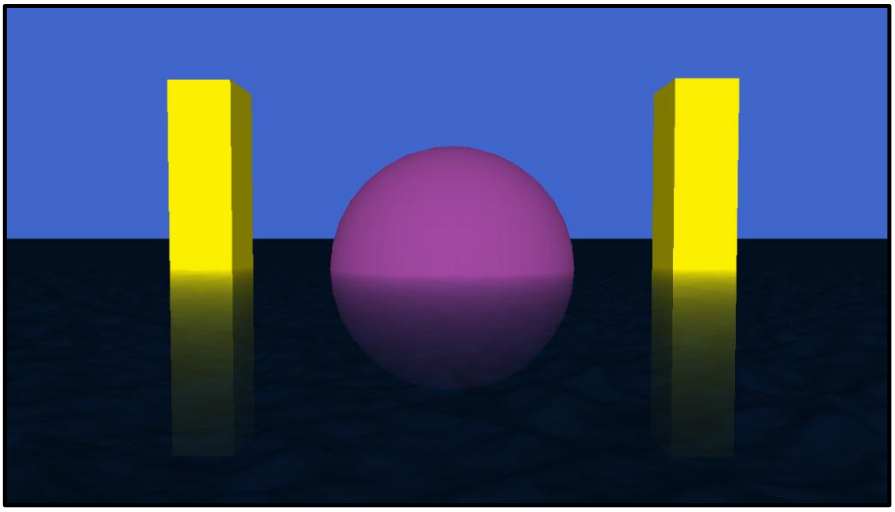

Figure 34: Output image of ocean surface with underwater effect.

© DigiPen Institute of Technology Europe - Bilbao

When the camera is submerged, an additional depth-based term is applied to account for light attenuation at the camera position. This term is computed in the same way as the depth-based absorption but using the camera's world-space vertical position (C_y) as depth.

The final absorption becomes:

$$f = 1 - (1 - f_{\mathrm{h}}) \cdot (1 - f_{\mathrm{d}}) \cdot (1 - f_{\mathrm{c}})$$

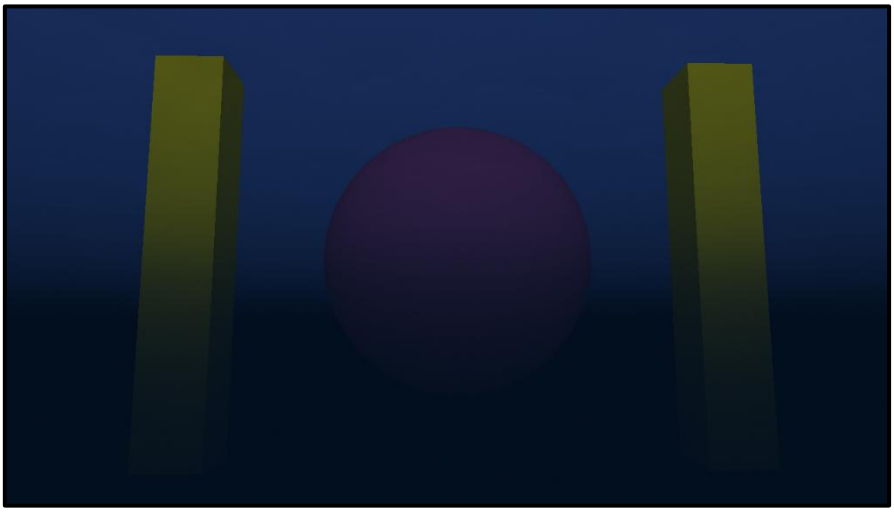

Figure 35: Output image of the underwater view with underwater effect.

This approach mimics light attenuation along the path from the water surface to the camera while ensuring that shallower geometry receives less absorption.

© DigiPen Institute of Technology Europe - Bilbao

5.6.1. Transition Across the Water Surface

Determining whether a pixel lies above or below the water surface using explicit geometric tests would be computationally expensive. Instead, a screen-space blending approach is used to create a smooth transition between above water and underwater rendering.

First, a binary indicator is defined to classify whether the camera is above the water surface using the camera's world space height (C_y). Assuming the surface is at y = 0:

$$C_{above} = step(0, C_y)$$

To avoid hard discontinuity at the water plane, two smooth transition functions are defined using a band of thickness l, centered around the water surface that check the world space height (P_y) of the fragment:

$$B_{under} = smoothstep(-l, 0, P_y)$$
$$B_{above} = smoothstep(0, l, P_y)$$

Since the appropriate transition depends on whether the camera is above or below the surface, the final blending factor is obtained by choosing between both bands:

$$\mathrm{I} = \mathrm{mix}(B_{above}, B_{under}, C_{above})$$

This factor represents the amount of underwater effect applied at a given camera height, ensuring a smooth transition between above water and underwater rendering. Pixels within the blending band receive a partial contribution from both states, while pixels well above the band or below it are dominated by their respective effects.

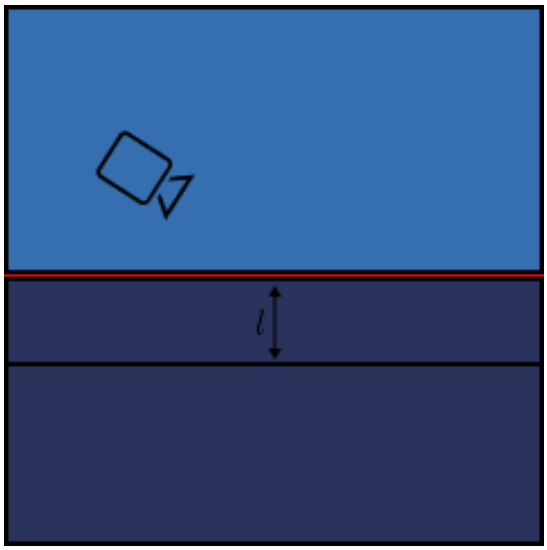

Figure 37: Blending band under the water surface.

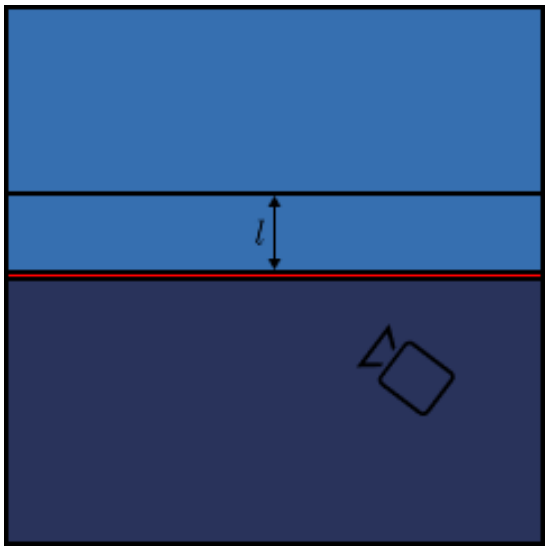

Figure 36: Blending band above the water surface.

© DigiPen Institute of Technology Europe - Bilbao

5.7. Sun

The directional light is used to render the sun in the scene, based on its direction relative to the camera. The sun intensity is computed as:

$$I(v) = smoothstep(\theta_{max} - \epsilon, \theta_{max}, v \cdot d)$$

where:

- v: view direction.
- d: directional light direction.
- θ_{max}: angular radius of the sun.
- ϵ: controls the smoothness of the sun's edge.

Using this formulation, the sun appears as a circular highlight in the sky, with intensity decreasing smoothly near the edges, producing a natural and visually pleasing effect.

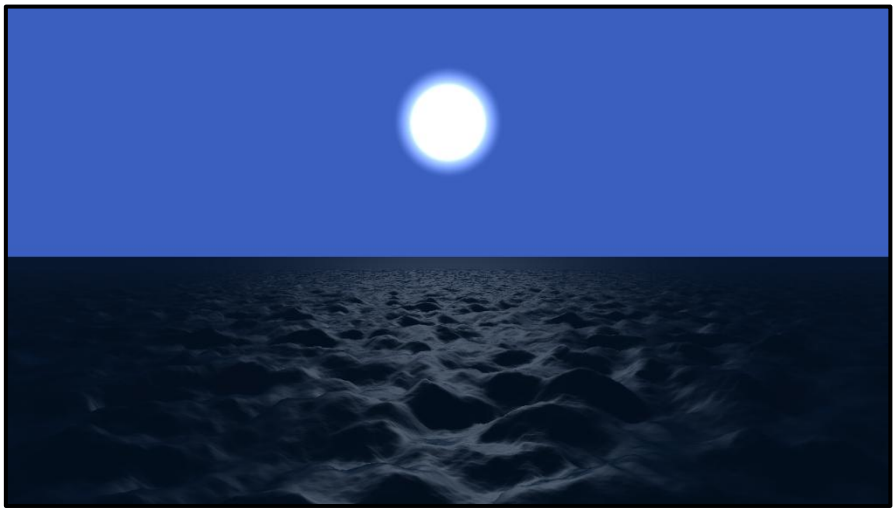

Figure 38: Rendered sun based on directional light.

© DigiPen Institute of Technology Europe - Bilbao

5.8. Final render

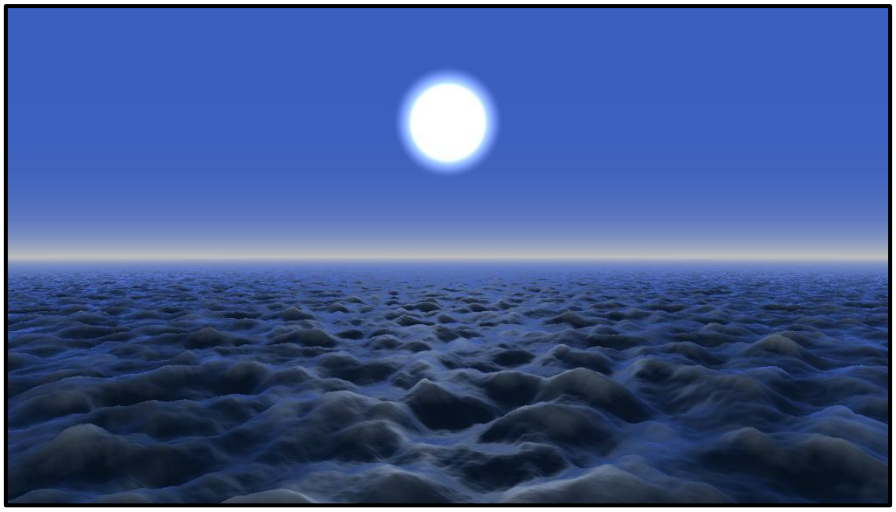

Figure 41: Output image of ocean surface with visual enhancements.

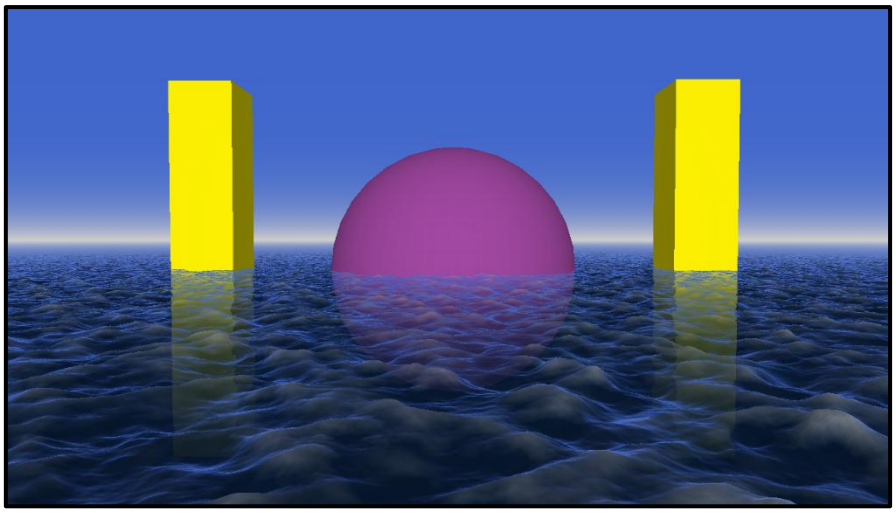

Figure 39: Output image of submerged objects with visual enhancements.

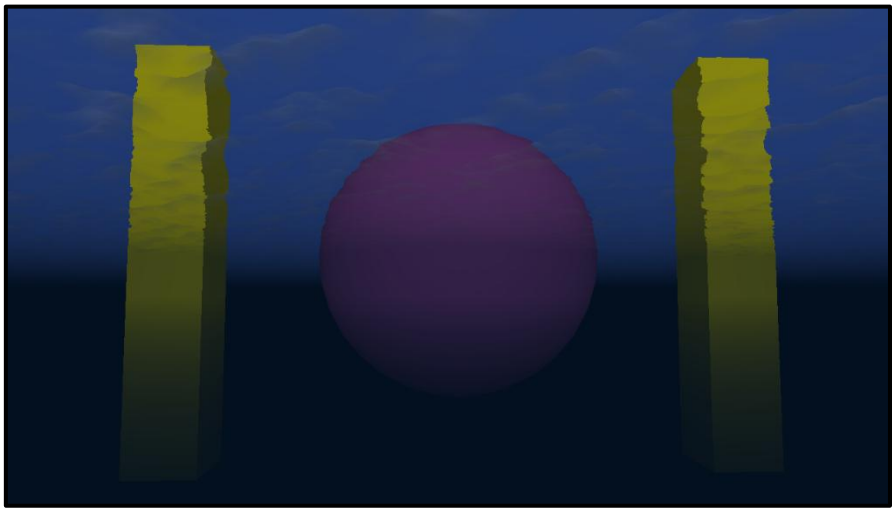

Figure 40: Output image of underwater view with visual enhancements.

© DigiPen Institute of Technology Europe - Bilbao

Results

The combination of the configurable visual effects described in the previous sections enables the creation of complex and realistic ocean scenes under a variety of conditions. By adjusting parameters, the rendering system can simulate diverse scenarios ranging from calm seas to stormy weather.

The following figures show example scenes produced with different parameter configurations, demonstrating the flexibility and visual richness achievable with the system:

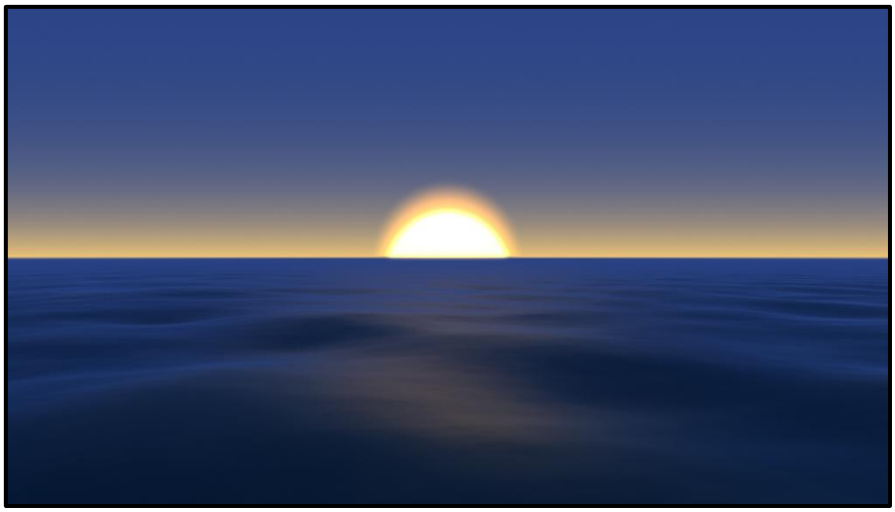

Figure 43: Calm ocean in the morning, with gentle waves, and soft atmospheric lighting.

Figure 42: Stormy night with strong waves, dense foam, and heavy fog reducing visibility.

© DigiPen Institute of Technology Europe - Bilbao

Limitations

This implementation has two clear limitations, both related to resolution, one concerning the procedural textures and the other the projected grid used for the ocean surface.

1. Projected grid resolution

The visual fidelity of the ocean surface is limited by the resolution of the projected grid used to represent the mesh. When the grid resolution is low the surface lacks smoothness.

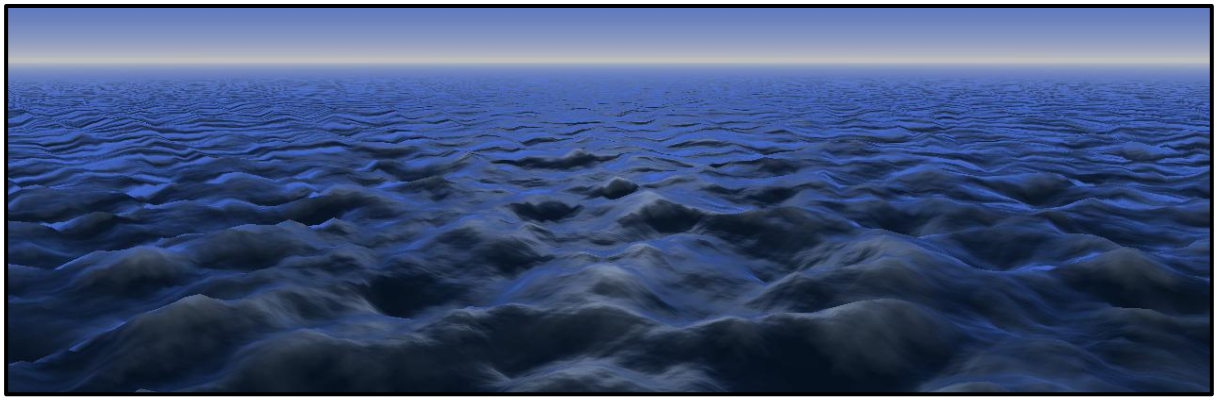

Figure 44: Output with low resolution projected grid, resulting in a surface lacking smoothness.

Additionally, camera movements can produce noticeable popping or jittering artifacts.

2. Texture resolution

The level of detail achievable for the ocean surface is directly limited by the resolution of the procedural textures used.

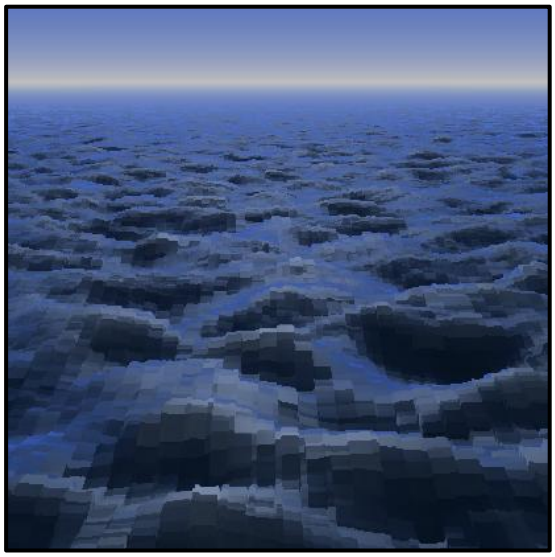

Figure 46: Output with low resolution textures and nearest sampling filter, producing a blocky appearance.

Figure 45: Output with high resolution texture. Fine wave details are captured, but small-scale water features produce visible artifacts.

Very high-resolution water detail may exceed feasible texture sizes. Attempting to capture extremely small-scale waves could require textures that exceed the practical GPU memory limit, potentially limiting visual fidelity in scenarios that demand very detailed water.

© DigiPen Institute of Technology Europe - Bilbao

Conclusions

The implementation presented demonstrates a robust and efficient approach to real-time infinite ocean surface rendering. By combining a projected grid with tileable procedural texture generation in GPU, the system keeps the computational load low while maintaining high visual fidelity, making it well-suited for interactive applications.

This design cleverly minimizes per-vertex computations across the entire ocean surface, trading off increased texture memory and sampling, while still enabling the simulation of large-scale water with complex wave interactions at acceptable frame rates, which is critical for real-time applications.

The procedural generation of waves introduces a wide range of configurable parameters, enabling fine control over wave behavior. However, this also makes the system challenging to understand at first, as the large number of parameters and the way they influence the simulation are not always intuitive. As a result, predicting how changes will affect the final appearance can be difficult initially. Nevertheless, once these relationships are understood, the implementation provides a powerful and highly flexible toolset.

In addition to the core simulation, a set of visual enhancement techniques has been integrated to improve realism and visual coherence. These include ocean surface and atmospheric effects that extend the basic geometric representation into a more physically accurate rendering, allowing the ocean to interact more convincingly with light, the environment, and the viewer.

In summary, this approach represents a clever and practical compromise between performance and visual quality, offering a strong foundation for further enhancements.

© DigiPen Institute of Technology Europe - Bilbao

How to use the Demo

Running the application launches a default ocean scene, allowing the user to interactively explore and modify the simulation using the *Dear ImGui* user interface. The interface exposes all the parameters controlling the ocean, lighting, atmosphere, and scene presets, providing a flexible environment to experiment with different configurations in real time.

Figure 47: Application user interface.

The *Ocean Settings* panel exposes all parameters related to the ocean surface. This includes the projected grid configuration, procedural texture generation, wave properties, foam behavior, and surface shading parameters such as lighting, reflection, and refraction.

Figure 48: Ocean settings panel of application user interface.

© DigiPen Institute of Technology Europe - Bilbao

The *Atmosphere Settings* panel provides control over the environmental aspects of the scene. This includes sky color, fog parameters, underwater effects, blending across the water surface, and sun properties.

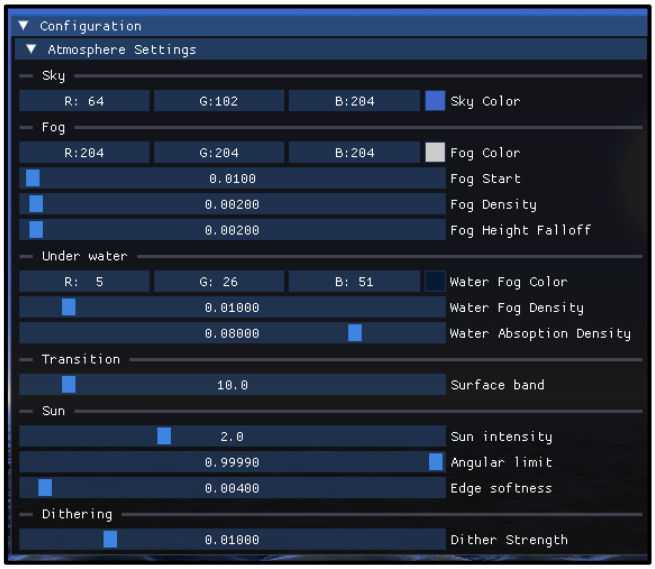

Figure 49: Atmosphere settings panel of application user interface.

Finally, the *Scene Presets* system allows saving and loading predefined configurations. Presets are stored in the directory *data/ocean_presets/* and are automatically detected by the interface. Users can save the current state of the scene as a new preset or load existing ones from this directory. When switching between presets, an animation is applied that smoothly interpolates all parameters over time, allowing gradual transitions between different ocean conditions instead of abrupt changes.

Figure 50: Scene Presets settings panel of application user interface.

© DigiPen Institute of Technology Europe - Bilbao

Bibliography

Finch, M. (2004). Effective Water Simulation from Physical Models. In NVIDIA, *GPU Gems.* Retrieved from developer.nvidia.com/gpugems/gpugems/part-i-natural-effects/chapter-1-effective-water-simulation-physical-models

Johanson, C. (2004). *Real-time water rendering: Introducing the projected grid concept.* Lund University, Department of Computer Science.

NVIDIA Corporation. (n.d.). *Ocean Surface Simulation.* Retrieved from https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/OceanCS_Slides.pdf

Schlick, C. (1994). *An Inexpensive BRDF Model for Physically-based endering.*

Tessendorf, J. (2001). Simulating ocean water. *SIGGRAPH 2001 Course Notes (Course 47).* ACM SIGGRAPH.

Thomas, D., & Eric, H. (2019). Procedural Stochastic Textures by Tiling and Blending. In W. Engel, *GPU Zen 2: Advanced Rendering Techniques.*

Ubisoft La Forge. (2024, July 25). *Making Waves in Ocean Surface Rendering using Tiling and Blending*. Retrieved from Ubisoft La Forge: https://www.ubisoft.com/en-us/studio/laforge/news/5WHMK3tLGMGsqhxmWls1Jw/making-waves-in-ocean-surface-rendering-using-tiling-and-blending

Wikipedia contributors. (n.d.). *Trochoidal wave*. Retrieved from Wikipedia: https://en.wikipedia.org/wiki/Trochoidal_wave

© DigiPen Institute of Technology Europe - Bilbao

Appendix – GLSL Mathematical Functions

This appendix summarizes a set of mathematical functions used in the rendering system GLSL (OpenGL Shading Language) shaders.

Step Function

The *step* function is a discontinuous binary function used to classify values based on a threshold. It is defined as:

$$step(edge, x) = \begin{cases} 0 & x < edge \\ 1 & x \geq edge \end{cases}$$

This function is commonly used to perform efficient binary classification, such as determining whether a value lies above or below a given threshold.

Smoothstep Function

The *smoothstep* function performs Hermite interpolation between two edge values, producing a smooth transition from 0 to 1 across the interval. It can be expressed mathematically as:

$$smoothstep(edge_0, edge_1, x) = \begin{cases} 0 & x \leq \text{edge}_0 \\ 1 & x \geq \text{edge}_1 \\ t^2 \cdot (3 - 2t) & \text{otherwise, with } t = \dfrac{x - \text{edge}_0}{\text{edge}_1 - \text{edge}_0} \end{cases}$$

© DigiPen Institute of Technology Europe - Bilbao